



iOS 底层探索 - KVO



leejunhui

2020-02-24

阅读 30 分钟



1



iOS 底层探索系列

- iOS 底层探索 - alloc & init
- iOS 底层探索 - calloc 和 isa
- iOS 底层探索 - 类
- iOS 底层探索 - cache_t
- iOS 底层探索 - 方法
- iOS 底层探索 - 消息查找
- iOS 底层探索 - 消息转发
- iOS 底层探索 - 应用加载
- iOS 底层探索 - 类的加载
- iOS 底层探索 - 分类的加载
- iOS 底层探索 - 类拓展和关联对象
- iOS 底层探索 - KVC
- iOS 底层探索 - KVO

一、KV...

1...

1...

1...

1...

二、KV...

2...

2...

2...

2...

2...

三、自...

~

在 **Objective-C** 和 **Cocoa** 中，有许多事件之间进行通信的方式，并且每个都有不同程度的形式和耦合：

NSNotification & **NSNotificationCenter** 提供了一个中央枢纽，一个应用的任何部分都可能通知或者被通知应用的其他部分的变化。唯一需要做的是要知道在寻找什么，主要是通知的名字。例如，

UIApplicationDidReceiveMemoryWarningNotification 是给应用发了一个内存不足的信号。

Key-Value Observing 键值观察通过侦听特定键路

径上的更改，可以在特定对象实例之间进行特殊的事件自省。例如：一个 `ProgressView` 可以观察网络请求的 `numberOfBytesRead` 来更新它自己的 `progress` 属性。

`Delegate` 是一个流行的传递事件的设计模式，通过定义一系列的方法来传递给指定的处理对象。例如：`UIScrollView` 每次它的 `scroll offset` 改变的时候都会发送 `scrollViewDidScroll:` 到它的代理

`Callbacks` 不管是像 `NSOperation` 里的 `completionBlock`（当 `isFinished==YES` 的时候会触发），还是 `C` 里边的函数指针，传递一个函数钩子比如

`SCNetworkReachabilitySetCallback(3)`。

一、KVO 初探

根据苹果官方文档的定义，`KVO` (Key Value Observing) 键值观察是建立在 `KVC` 基础之上的，所以如果对 `KVC` 不是很了解的读者可以查看上一篇 `KVC` 底层探索的文章。

我相信大多数开发者应该对于 `KVO` 都能熟练掌握，不过我们还是回顾一下官网对于 `KVO` 的解释吧。

1.1 什么是 `KVO`?

`KVO` 提供了一种当其他对象的属性发生变化就会通知观察者对象的机制。根据官网的定义，属性的分类可以分为下列三种：

- **Attributes:** 简单属性，比如基本数据类型，字符串和布尔值，而诸如 `NSNumber` 和其它一些不可变类型比如 `NSColor` 也可以被认为是简单属性
- **To-one relationships:** 这些是具有自己属性的可

变对象属性。即对象的属性可以更改，而无需更改对象本身。例如，一个 **Account** 对象可能具有一个 **owner** 属性，该属性是 **Person** 对象的实例，而 **Person** 对象本身具有 **address** 属性。**owner** 的地址可以更改，但却无需更改 **Account** 持有的 **owner** 属性。也就是说 **Account** 的 **owner** 属性未被更改，只是 **address** 被更改了。

- **To-many relationships:** 这些是集合对象属性。尽管也可以使用自定义集合类，但是通常使用 **NSArray** 或 **NSSet** 的实例来持有此集合。

而 **KVO** 对于这三种属性都能适用。下面举一个例子：

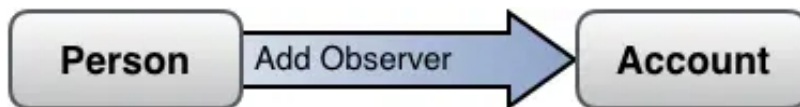


如上所示，**Person** 对象有一个 **Account** 属性，而 **Account** 对象又有 **balance** 和 **interestRate** 两个属性。并且这两个属性对于 **Person** 对象来说都是可读写的。如果想实现一个功能：当余额或利率变化的时候需要通知到用户。一般来说可以使用轮询的方式，**Person** 对象定期从 **Account** 属性中取出 **balance** 和 **interestRate**。但这种方式是效率低下且不切实际的，更好的方式是使用 **KVO**，类似于余额或利率变动时，**Person** 对象收到了通知一样。

要实现 **KVO** 的前提是要确保被观察对象是符合 **KVO** 机制的。一般来说，继承于 **NSObject** 根类的对象及其属性都自动符合 **KVO** 这一机制。当然也可以自己去实现 **KVO** 符合。也就是说实际上 **KVO** 机制分为**自动符合**和**手动符合**。

一旦确定了对象和属性是 **KVO** 符合的话，就需要历经三个步骤：

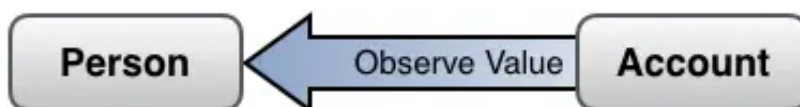
- 观察者注册



Person 对象需要将自己注册到 **Account** 的某一个具体属性上。这个过程是通过

addObserver:forKeyPath:options:context: 实现的，这个方法需要指定监听者(**observer**)、监听谁(**keypath**)、监听策略(**options**)、监听上下文(**context**)

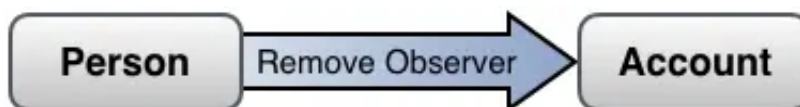
- 被观察者触发回调



Person 对象要接收 **Account** 被监听属性改动后发出的通知，需要自身实现

observeValueForKeyPath:ofObject:change:context: 方法来接收通知。

- 观察者取消注册



在观察者不需要再监听或自身生命周期结束的时候，需要取消注册。具体实现是通过向被观察对象发出 **removeObserver:forKeyPath:** 消息。

KVO 机制的最大好处你不需要自己去实现一个机制来获取对象属性何时改变以及改变后的结果。

1.2 KVO 三大流程解析

1.2.1 观察者注册

```
- (void)addObserver:(NSObject *)observer
    forKeyPath:(NSString *)keyPath
    options:(NSKeyValueObservingOptions)options
    context:(void *)context
```

observer:注册 KVO 通知的对象。观察者必须实现 key-value observing 方法

observeValueForKeyPath:ofObject:change:context:。

keyPath:被观察者的属性的 keypath，相对于接受者，值不能是 nil。

options: NSKeyValueObservingOptions 的组合，它指定了观察通知中包含了什么

context:在

observeValueForKeyPath:ofObject:change:context: 传给 **observer** 参数的上下文

前两个参数很好理解，而 **options** 和 **context** 参数则需要额外注意。

options 代表 NSKeyValueObservingOptions 的位掩码，需要注意 NSKeyValueObservingOptionNew & NSKeyValueObservingOptionOld，因为这些是你经常要用到的，可以跳过

NSKeyValueObservingOptionInitial &

NSKeyValueObservingOptionPrior:。

NSKeyValueObservingOptionNew: 表明通知中的更改字典应该提供新的属性值，如何可以的话。

NSKeyValueObservingOptionOld: 表明通知中的更改字典应该包含旧的属性值，如何可以的话。

NSKeyValueObservingOptionInitial: 这个枚举值比较特殊，如果指定了这个枚举值，在属性发生变化后立即通知观察者，这个过程甚至早于观察者注册。如果在注册的时候配置了

NSKeyValueObservingOptionNew，那么在通知的

更改字典中也会包含 `NSKeyValueChangeNewKey`，但是不会包括 `NSKeyValueChangeOldKey`。（在初始通知中，观察到的属性值可能是旧的，但是对于观察者来说是新的）其实简单来说就是这个枚举值会在属性变化前先触发一次

`observeValueForKeyPath` 回调。

`NSKeyValueObservingOptionPrior`: 这个枚举值会先后连续出发两次 `observeValueForKeyPath` 回调。同时在回调中的可变字典中会有一个布尔值的 `key - notificationIsPrior` 来标识属性值是变化前还是变化后的。如果是变化后的回调，那么可变字典中就只有 `new` 的值了，如果同时制定了

`NSKeyValueObservingOptionNew` 的话。如果你需要启动手动 KVO 的话，你可以指定这个枚举值然后通过 `willChange` 实例方法来观察属性值。在出发 `observeValueForKeyPath` 回调后再去调用

`willChange` 可能就太晚了。

这些选项允许一个对象在发生变化的前后获取值。在实践中，这不是必须的，因为从当前属性值获取的新值一般是可用的 也就是说

`NSKeyValueObservingOptionInitial` 对于在反馈 KVO 事件的时候减少代码路径是很有好处的。比如，如果你有一个方法，它能够动态的使一个基于 `text` 值的按钮有效，传

`NSKeyValueObservingOptionInitial` 可以使事件随着它的初始化状态触发一旦观察者被添加进去的话。

如何设置一个好的 `context` 值呢？这里有个建议：

```
static void * XXContext = &XXContext;
```

就是这么简单：一个静态变量存着它自己的指针。这意味着它自己什么也没有，使

`<NSKeyValueObserving>` 更完美。

我们简单测试一下在注册观察者时指定不同的枚举值会有怎么样的结果：

- 只指定 `NSKeyValueObservingOptionNew`

```
19 - (void)viewDidLoad
20 {
21     [super viewDidLoad];
22
23     JHPerson *person = [[JHPerson alloc] init];
24     self.person = person;
25     self.person.name = @"Kobe Bryant";
26     [self.person addObserver:self forKeyPath:@"name" options:NSKeyValueObservingOptionNew context:JHPersonNameContext];
27 }
28
29 - (void)touchesBegan:(NSSet<UITouch> *)touches withEvent:(UIEvent *)event
30 {
31     NSLog(@"来了 老弟");
32     self.person.name = @"Michael Jordan";
33 }
34
35 #pragma mark - KVO
36 - (void)observeValueForKeyPath:(NSString *)keyPath ofObject:(id)object change:(NSDictionary<NSKeyValueChangeKey, id> *)change
37     context:(void *)context
38 {
39     if (context == JHPersonNameContext) {
40         NSLog(@"改变数组 %@", change);
41         self.nameLabel.text = self.person.name;
42     } else {
43         [super observeValueForKeyPath:keyPath ofObject:object change:change context:context];
44     }
45 }
```

2020-02-17 16:24:20.642247+0800 KVOIndepth[80899:3662712] 来了 老弟
2020-02-17 16:24:22.422514+0800 KVOIndepth[80899:3662712] 改变数组 {
kind = 1;
new = "Michael Jordan";
}

- 只指定 `NSKeyValueObservingOptionOld`

```
19 - (void)viewDidLoad
20 {
21     [super viewDidLoad];
22
23     JHPerson *person = [[JHPerson alloc] init];
24     self.person = person;
25     self.person.name = @"Kobe Bryant";
26     [self.person addObserver:self forKeyPath:@"name" options:NSKeyValueObservingOptionOld context:JHPersonNameContext];
27 }
28
29 - (void)touchesBegan:(NSSet<UITouch> *)touches withEvent:(UIEvent *)event
30 {
31     NSLog(@"来了 老弟");
32     self.person.name = @"Michael Jordan";
33 }
34
35 #pragma mark - KVO
36 - (void)observeValueForKeyPath:(NSString *)keyPath ofObject:(id)object change:(NSDictionary<NSKeyValueChangeKey, id> *)change
37     context:(void *)context
38 {
39     if (context == JHPersonNameContext) {
40         NSLog(@"改变数组 %@", change);
41         self.nameLabel.text = self.person.name;
42     } else {
43         [super observeValueForKeyPath:keyPath ofObject:object change:change context:context];
44     }
45 }
```

2020-02-17 16:24:51.931087+0800 KVOIndepth[80811:3663562] 来了 老弟
2020-02-17 16:24:52.797391+0800 KVOIndepth[80811:3663562] 改变数组 {
kind = 1;
old = "Kobe Bryant";
}

- 指定 `NSKeyValueObservingOptionInitial`

```
19 - (void)viewDidLoad
20 {
21     [super viewDidLoad];
22
23     JHPerson *person = [[JHPerson alloc] init];
24     self.person = person;
25     self.person.name = @"Kobe Bryant";
26     [self.person addObserver:self forKeyPath:@"name" options:NSKeyValueObservingOptionInitial context:JHPersonNameContext];
27 }
28
29 - (void)touchesBegan:(NSSet<UITouch> *)touches withEvent:(UIEvent *)event
30 {
31     NSLog(@"来了 老弟");
32     self.person.name = @"Michael Jordan";
33 }
34
35 #pragma mark - KVO
36 - (void)observeValueForKeyPath:(NSString *)keyPath ofObject:(id)object change:(NSDictionary<NSKeyValueChangeKey, id> *)change
37     context:(void *)context
38 {
39     if (context == JHPersonNameContext) {
40         NSLog(@"改变数组 %@", change);
41         self.nameLabel.text = self.person.name;
42     } else {
43         [super observeValueForKeyPath:keyPath ofObject:object change:change context:context];
44     }
45 }
```

2020-02-17 16:25:23.892655+0800 KVOIndepth[80128:3664527] 改变数组 {
kind = 1;
}
2020-02-17 16:25:24.797912+0800 KVOIndepth[80128:3664527] 来了 老弟
2020-02-17 16:25:28.178693+0800 KVOIndepth[80128:3664527] 改变数组 {
kind = 1;
}

可以看到，只指定了

`NSKeyValueObservingOptionInitial` 后触发了两个回调，并且一次是在属性值变化前，一次是在属性值变化后。同时并且没有新值和旧值返回，我们加一个 `NSKeyValueObservingOptionNew` 和 `NSKeyValueObservingOptionOld`:

```
19 - (void)viewDidLoad
20 {
21     [super viewDidLoad];
22
23     JHPerson *person = [[JHPerson alloc] init];
24     self.person = person;
25     self.person.name = @"Kobe Bryant";
26     [self.person addObserver:self forKeyPath:@"name"
27      options:NSKeyValueObservingOptionInitial|NSKeyValueObservingOptionNew|NSKeyValueObservingOptionOld
28      context:JHPersonNameContext];
29 }
30
31 - (void)touchesBegan:(NSSet<UITouch * > *)touches withEvent:(UIEvent *)event
32 {
33     NSLog(@"来了 老弟");
34     self.person.name = @"Michael Jordan";
35 }
36
37 #pragma mark - KVO
38 - (void)observeValueForKeyPath:(NSString *)keyPath ofObject:(id)object change:(NSDictionary<NSKeyValueChangeKey, id> *)change context:(void *)context
39 {
40     if (context == JHPersonNameContext) {
41         NSLog(@"改变数组 %@", change);
42         self.nameLabel.text = self.person.name;
43     } else {
44         [super observeValueForKeyPath:keyPath ofObject:object change:change context:context];
45     }
46 }
```

2020-02-17 16:27:03.472188+0800 KVOInDepth[80144:3666251] 改变数组 {
kind = 1;
new = "Kobe Bryant";
}
2020-02-17 16:27:06.063724+0800 KVOInDepth[80144:3666251] 来了 老弟
2020-02-17 16:27:07.054460+0800 KVOInDepth[80144:3666251] 改变数组 {
kind = 1;
new = "Michael Jordan";
old = "Kobe Bryant";
}

在我们加上新值和旧值的枚举之后，新值在两次回调后被返回，但是第一次的新值其实是最开始的属性值，第二次才是改变之后的属性值，而旧值在第二次真正属性值被改变后返回。

- 指定 `NSKeyValueObservingOptionPrior`

```
19 - (void)viewDidLoad
20 {
21     [super viewDidLoad];
22
23     JHPerson *person = [[JHPerson alloc] init];
24     self.person = person;
25     self.person.name = @"Kobe Bryant";
26     [self.person addObserver:self forKeyPath:@"name" options:NSKeyValueObservingOptionPrior context:JHPersonNameContext];
27 }
28
29 - (void)touchesBegan:(NSSet<UITouch * > *)touches withEvent:(UIEvent *)event
30 {
31     NSLog(@"来了 老弟");
32     self.person.name = @"Michael Jordan";
33 }
34
35 #pragma mark - KVO
36 - (void)observeValueForKeyPath:(NSString *)keyPath ofObject:(id)object change:(NSDictionary<NSKeyValueChangeKey, id> *)change context:(void *)context
37 {
38     if (context == JHPersonNameContext) {
39         NSLog(@"改变数组 %@", change);
40         self.nameLabel.text = self.person.name;
41     } else {
42         [super observeValueForKeyPath:keyPath ofObject:object change:change context:context];
43     }
44 }
45
46 @end
```

2020-02-17 16:29:41.122213+0800 KVOInDepth[80164:3668351] 来了 老弟
2020-02-17 16:29:43.063572+0800 KVOInDepth[80164:3668351] 改变数组 {
kind = 1;
notificationIsPrior = 1;
}
2020-02-17 16:29:43.883991+0800 KVOInDepth[80164:3668351] 改变数组 {
kind = 1;
}

可以看到，`NSKeyValueObservingOptionPrior` 枚举值是在属性值发生变化后触发了两次回调，同时也没有新值和旧值的返回，我们加一个 `NSKeyValueObservingOptionNew` 和 `NSKeyValueObservingOptionOld`:

```
19 - (void)viewDidLoad
20 {
21     [super viewDidLoad];
22
23     JHPerson *person = [[JHPerson alloc] init];
24     self.person = person;
25     self.person.name = @"Kobe Bryant";
26     [self.person addObserver:self forKeyPath:@"name"
27         options:NSKeyValueObservingOptionPrior|NSKeyValueObservingOptionNew|NSKeyValueObservingOptionOld
28         context:JHPersonNameContext];
29 }
30
31 - (void)touchesBegan:(NSSet<UITouch> *)touches withEvent:(UIEvent *)event
32 {
33     NSLog(@"来了 弟弟");
34     self.person.name = @"Michael Jordan";
35 }
36
37 #pragma mark - KVO
38 - (void)observeValueForKeyPath:(NSString *)keyPath ofObject:(id)object change:(NSDictionary<NSKeyValueChangeKey, id> *)change context:(void *)context
39 {
40     if (context == JHPersonNameContext) {
41         NSLog(@"改变数组 %@", change);
42         self.nameLabel.text = self.person.name;
43     } else {
44         [super observeValueForKeyPath:keyPath ofObject:object change:change context:context];
45     }
46 }
```

```
2020-02-17 16:31:38.126203+0800 KVOInDepth[80182:3670145] 来了 弟弟
2020-02-17 16:31:39.739639+0800 KVOInDepth[80182:3670145] 改变数组 {
    kind = 1;
    notificationsIsPrior = 1;
    old = "Kobe Bryant";
}
2020-02-17 16:31:41.483892+0800 KVOInDepth[80182:3670145] 改变数组 {
    kind = 1;
    new = "Michael Jordan";
    old = "Kobe Bryant";
}
```

可以看到，在第一次回调里没有新值，第二次才有，而旧值在两次回调里面都有。

- keyPath 字符串问题

我们在注册观察者的时候，要求传入的 `keyPath` 是字符串类型，如果我们拼写错误的话，编译器是不能帮我们检查出来的，所有最佳实践应该是使用 `NSStringFromSelector(SEL aSelector)`，比如我们要观察 `tableView` 的 `contentSize` 属性，我们可以这样使用：

```
NSStringFromSelector(@selector(contentSize
```

1.2.2 观察者接收通知

```
- (void)observeValueForKeyPath:(NSString *)  
    ofObject:(id)object  
    change:(NSDictionary  
    context:(void *)con
```

这个方法就是观察者接收通知的地方，除了 `change` 参数之外，其他三个参数都与观察者注册的时候传入的三个参数一一对应。

- 不同对象监听相同的 `keypath`

默认情况下，我们在

`addObserver:forKeyPath:options:context:` 方法的最后一个参数传入的是 `NULL`，因为这个方法签名中最后一个参数 `context` 是 `void *`，所以需要传入一个空指针，而根据下图我们可知，`nil` 只是一个对象的字面零值，这里需要的是一个指针，所以需要传 `NULL`。

标志	值	含义
<code>NULL</code>	<code>(void *)0</code>	C指针的字面零值
<code>nil</code>	<code>(id)0</code>	Objective-C对象的字面零值
<code>Nil</code>	<code>(Class)0</code>	Objective-C类的字面零值
<code>NSNull</code>	<code>[NSNull null]</code>	用来表示零值的单独的对象

但是如果是不同的对象都监听同一属性，我们就需要给 `context` 传入一个可以区分不同对象的字符串指针：

```
static void *StudentNameContext = &Student  
static void *PersonNameContext = &PersonNa  
  
[self.person addObserver:self forKeyPath:@  
[self.student addObserver:self forKeyPath:
```

```

- (void)observeValueForKeyPath:(NSString *)
    keyPath:
    ofObject:(id)object
    change:(NSDictionary *)change
    context:(void *)context {
    if (context == PersonNameContext) {
        // ...
    } else if (context == StudentNameContext) {
        // ...
    }
}

```

- 需要自己处理 `superclass` 的 `observe` 事务

对于 `Objective-C`，很多时候 `Runtime` 系统都会自动帮助处理 `superclass` 的方法。譬如对于 `dealloc`，假设类 `Father` 继承自 `NSObject`，而类 `Son` 继承自 `Father`，创建一个 `Son` 的实例 `aSon`，在 `aSon` 被释放的时候，`Runtime` 会先调用 `Son#dealloc`，之后会自动调用 `Father#dealloc`，而无需在 `Son#dealloc` 中显式执行 `[super dealloc]`；。但 `KVO` 不会这样，所以为了保证父类（父类可能也会自己 `observe` 事务要处理）的 `observe` 事务也能被处理。

```

- (void)observeValueForKeyPath:(NSString *)keyPath:
    ofObject:(id)object
    change:(NSDictionary *)change
    context:(void *)context {
    if (object == _tableView && [keyPath isEqualToString:@"text"]) {
        [self configureView];
    } else {
        [super observeValueForKeyPath:keyPath:
            ofObject:object
            change:change
            context:context];
    }
}

```

1.2.3 取消注册

- (void)removeObserver:(NSObject *)observe
- (void)removeObserver:(NSObject *)observe

取消注册有两个方法，不过建议还是跟注册和通知两个流程统一，选用带有 `context` 参数的方法。

- 取消注册与注册是一对一的关系

一旦对某个对象上的属性注册了键值观察，可以选择在收到属性值变化后取消注册，也可以在观察者声明周期结束之前(比如：`dealloc` 方法)取消注册，如果忘记调用取消注册方法，那么一旦观察者被销毁后，**KVO** 机制会给一个不存在的对象发送变化回调消息导致野指针错误。

- 不能重复取消注册

取消注册也不能对同一个观察者重复多次，为了避免 `crash`，可以把取消注册的代码包裹在 `try&catch` 代码块中：

```
static void * ContentSizeContext = &Cont

- (void)viewDidLoad {

    [super viewDidLoad];

    // 1. subscribe
    [_tableView addObserver:self
                    forKeyPath:NSStringFrom
                    options:NSKeyValueOb
                    context:ContentSizeC
    }

    // 2. responding
    - (void)observeValueForKeyPath:(NSString
                                ofObject:(id)objec
                                change:(NSDictio
                                context:(void *)c
```

```
        if (context == ContentSizeContext) {  
            // configure view  
        } else {  
            [super observeValueForKeyPath:key  
        }  
    }  
  
- (void)dealloc {
```

1.3 "自动挡" 和 "手动挡"

默认情况下，我们只需要按照前面说的 **三步曲** 的方式来实现对属性的键值观察，不过这属于是「自动挡」，什么意思呢？就是说属性值变化完全是由系统控制，我们只需要告诉系统监听什么属性，然后就直接等系统告诉我们就完事了。而实际上，**KVO** 还支持「手动挡」。

要让系统知道我们想开启手动挡，需要修改类方法 **automaticallyNotifiesObserversForKey:** 的返回值，这个方法如果返回 **YES** 就是自动挡，返回 **NO** 就是手动挡。同时该类方法还能精准实策，让我们选择对哪些属性是自动，哪些属性是手动。

```
+ (BOOL)automaticallyNotifiesObserversForKey  
  
    BOOL automatic = NO;  
    if ([theKey isEqualToString:@"balance"]  
        automatic = NO;  
    }  
    else {  
        automatic = [super automaticallyNo  
    }  
    return automatic;  
}
```

同样的，如上代码所示，我们使用 **automaticallyNotifiesObserversForKey** 的最佳实

践仍然需要把我们需要手动或自动的代码排除后去调用下父类的方法来确保不会有出现问题。

- 自动 KVO 触发方式

```
// Call the accessor method.
[account setName:@"Savings"];

// Use setValue:forKey:.
[account setValue:@"Savings" forKey:@"name"];

// Use a key path, where 'account' is a kv
[document setValue:@"Savings" forKeyPath:@"account.name"];

// Use mutableArrayValueForKey: to retrieve an array.
Transaction *newTransaction = <#Create a new transaction.
NSMutableArray *transactions = [account mutableArrayValueForKey:@"transactions"];
[transactions addObject:newTransaction];
```

如上代码所示是自动 KVO 的触发方式

- 手动 KVO 触发方式

其实手动 KVO 可以帮助我们将多个属性值的更改合并成一个，这样在回调的时候就有一次了，同时也能最大程度地减少处于应用程序特定原因而导致的通知发生。

```
- (void)setBalance:(double)theBalance {
    [self willChangeValueForKey:@"balance"];
    _balance = theBalance;
    [self didChangeValueForKey:@"balance"];
}
```

如上代码所示，最朴素的手动 KVO 使用方法就是在属性值改变前对观察者发送 `willChangeValueForKey` 实例方法，在属性值改变之后对观察者发送

`didChangeValueForKey` 实例方法，参数都是所观察的键。

当然，上面这种方式不是最佳的，为了性能最佳，可以在属性的 `setter` 中判断是否要执行 `will + did`:

```
- (void)setBalance:(double)theBalance {
    if (theBalance != _balance) {
        [self willChangeValueForKey:@"balance"];
        _balance = theBalance;
        [self didChangeValueForKey:@"balance"];
    }
}
```

但是，如果对一个属性的改变会影响到多个键的话，则需要如下的操作：

```
- (void)setBalance:(double)theBalance {
    [self willChangeValueForKey:@"balance"];
    [self willChangeValueForKey:@"itemChanged"];
    _balance = theBalance;
    _itemChanged = _itemChanged + 1;
    [self didChangeValueForKey:@"itemChanged"];
    [self didChangeValueForKey:@"balance"];
}
```

对于有序的一对多关系属性，不仅必须指定已更改的键，还必须指定更改的类型和所涉及对象的索引。更改的类型是 `NSKeyValueChange`，它指定 `NSKeyValueChangeInsertion`，`NSKeyValueChangeRemoval` 或 `NSKeyValueChangeReplacement`，受影响的对象的索引作为 `NSIndexSet` 对象传递：

```
- (void)removeTransactionsAtIndexes:(NSIndexSet *)indexes {
    [self willChange:NSKeyValueChangeRemoval
        valuesAtIndexes:indexes forKey:@"transactions"];
}
```



```
// Remove the transaction objects at the  
  
[self didChange:NSKeyValueChangeRemoval  
    valuesAtIndexes:indexes forKey:@"tra  
}
```

1.4 注册从属关系的 KVO

所谓从属关系，指的是一个对象的某个属性的值取决于另一个对象的一个或多个属性。对于不同类型的属性，有不同的方式来实现。

- 一对一关系

要触发 一对一 类型属性的自动 KVO，有两种方式。一种是重写 `keyPathsForValuesAffectingValueForKey` 方法，一种是实现一个合适的方法。

```
- (NSString *)fullName {  
    return [NSString stringWithFormat:@"%@"  
}
```

比如上面的代码，`fullName` 由 `firstName` 和 `lastName` 组成，所以重写 `fullName` 属性的 `getter` 方法。这样，不论是 `firstName` 还是 `lastName` 发生了改变，监听 `fullName` 属性的观察者都会收到通知。

```
+ (NSSet *)keyPathsForValuesAffectingValue  
  
    NSSet *keyPaths = [super keyPathsForVa  
  
    if ([key isEqualToString:@"fullName"])  
        NSArray *affectingKeys = @[@"lastN  
        keyPaths = [keyPaths setByAddingOb
```

```
    }  
    return keyPaths;  
}
```

如上代码所示，通过实现类方法 `keyPathsForValuesAffectingValueForKey` 来返回一个集合。值得注意的是，这里需要先对父类发送 `keyPathsForValuesAffectingValueForKey` 消息，以免干扰父类中对此方法的重写。

实际上还有一个便利的方法，就是 `keyPathsForValuesAffecting<Key>`，`Key` 是属性的名称（需要首字母大写）。这个方法的效果和 `keyPathsForValuesAffectingValueForKey` 是一样的，但针对的某个具体属性。

```
+ (NSSet *)keyPathsForValuesAffectingFullN  
    return [NSSet setWithObjects:@"lastName"  
}
```

相对来说，在分类中去使用 `keyPathsForValuesAffectingFullName` 更合理，因为分类中是不允许重载方法的，所以 `keyPathsForValuesAffectingValueForKey` 方法肯定是不能在分类中使用的。

- 一对多关系

`keyPathsForValuesAffectingValueForKey`：方法不支持包含一对多关系的 `Key Path`。例如，假设你有一个 `Department` 对象，该对象与 `Employee` 有一对多关系（即 `employees` 属性），而 `Employee` 具有 `salary` 属性。如果需要在 `Department` 对象上增加

`totalSalary` 属性，而该属性取决于关系中所有 `Employees` 的薪水。例如，您不能使用 `keyPathsForValuesAffectingTotalSalary` 和返回 `employees.salary` 作为键来执行此操作。

```
- (void)observeValueForKeyPath:(NSString *)keyPath
                        withObject:(id)object
                        forKeyPath:(NSString *)keyPath {
    if (context == totalSalaryContext) {
        [self updateTotalSalary];
    }
    else {
        // deal with other observations and/
    }
}

- (void)updateTotalSalary {
    [self setTotalSalary:[self valueForKeyPath:@"employees.salary"]];
}

- (void)setTotalSalary:(NSNumber *)newTotalSalary {
    if (totalSalary != newTotalSalary) {
        [self willChangeValueForKey:@"totalSalary"];
        _totalSalary = newTotalSalary;
        [self didChangeValueForKey:@"totalSalary"];
    }
}

- (NSNumber *)totalSalary {
    return _totalSalary;
}
```

如上代码所示，将 `Department` 实例对象注册为观察者，然后观察对象为 `totalSalary` 属性，但是在通知回调中会手动调用 `totalSalary` 属性的 `setter` 方法，并且传入值是通过 `KVC` 的集合运算符的方式取出 `employees` 属性所对应的集合中所有 `sum` 值之和。然后在 `totalSalary` 属性的 `setter` 方法中，会相应的调用 `willChangeValueForKey:` 和 `didChangeValueForKey:` 方法。

如果使用的是 `Core Data`，你还可以把 `Department` 注册到 `NSNotificationCenter` 中来作为托管对象上下文的观察者。`Department` 应以类似于观察键值的方式响应 `Employee` 发布的相关变更通知。

二、KVO 原理探究

Automatic key-value observing is implemented using a technique called isa-swizzling.

【译】自动的键值观察的实现基于 `isa-swizzling`。

The isa pointer, as the name suggests, points to the object's class which maintains a dispatch table. This dispatch table essentially contains pointers to the methods the class implements, among other data.

【译】`isa` 指针，顾名思义，指向的是对象所属的类，这个类维护了一个哈希表。这个哈希表基本上存储的是方法的 `SEL` 和 `IMP` 的键值对。

When an observer is registered for an attribute of an object the isa pointer of the observed object is modified, pointing to an intermediate class rather than at the true class. As a result the value of the isa pointer does not necessarily reflect the actual class of the instance.

【译】当一个观察者注册了对一个对象的某个属性键值观察之后，被观察对象的 `isa` 指针所指向的内容发生了变化，指向了一个中间类而不是真正的类。这也导致 `isa` 指针并不一定是指向实例所属的真正的类。

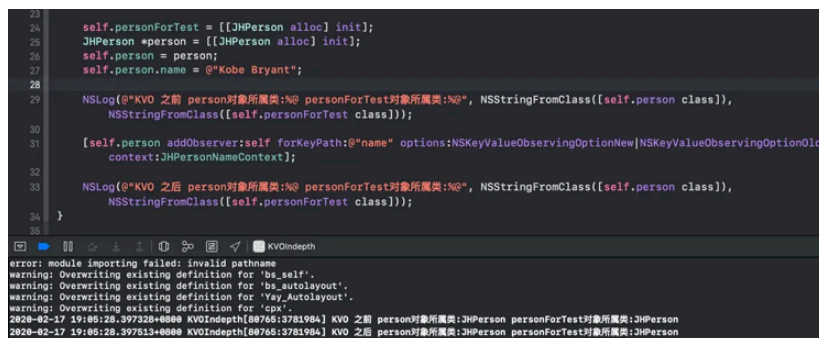
You should never rely on the isa pointer to determine class membership. Instead, you should use the class method to determine the class of an

object instance.

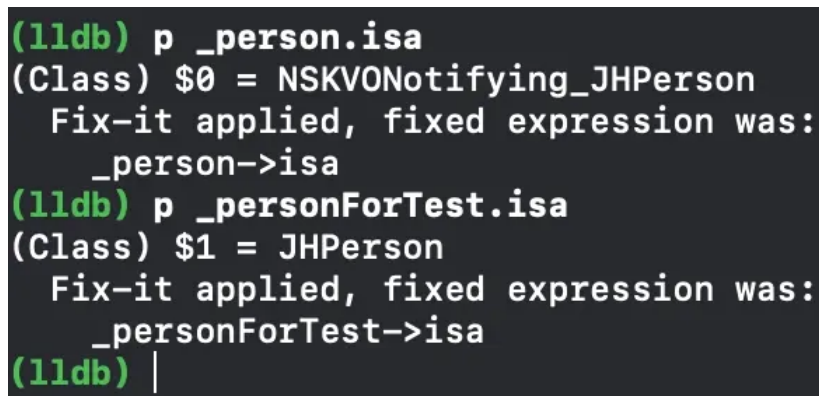
【译】你永远不应依靠 `isa` 指针来确定类成员身份。相反，你应该使用 `class` 方法来确定对象实例所属的类。

2.1 中间类

根据官网文档的内容，我们初步判断，在 `KVO` 底层实现中，会有一个所谓的中间类生成。而这个中间类会让对象的 `isa` 指针发生变化。我们不妨测试一下：



如上图所示，`person` 对象和 `personForTest` 对象都是属于 `JHPerson` 类的，而 `person` 对象又实现了 `KVO`，但是在控制台打印结果里面可以看到它们二者的类都是 `JHPerson` 类。不是说会有一个中间类生成吗？难道是这个中间类生成又被干掉了？我们直接 `LLDB` 大法测试一下：



Bingo~，所谓的中间类 `NSKVONotifying_JHPerson` 被我们找出来了。那么其实这里显然，系统是重写了中间类 `NSKVONotifying_JHPerson` 的 `class` 方法，

让我们以为对象的 `isa` 指针一直指向的都是 `JHPerson` 类。那么这个中间类和原来的类是什么关系呢?我们可以测试一下:

```
[self printClasses:[JHPerson class]];

[self.person addObserver:self forKeyPath:@"name" options:NSKeyValueObservingOptionNew|NSKeyValueObservingOptionOld
context:JHPersonNameContext];

[self printClasses:[JHPerson class]];
```

其中 `printClasses` 实现如下:

```
- (void)printClasses:(Class)cls{
    int count = objc_getClassList(NULL, 0)
    NSMutableArray *mArray = [NSMutableArray
    Class* classes = (Class*)malloc(sizeof
    objc_getClassList(classes, count);
    for (int i = 0; i<count; i++) {
        if (cls == class_getSuperclass(cls)
            [mArray addObject:classes[i]];
        }
    }
    free(classes);
    NSLog(@"classes = %@", mArray);
}
```

最终打印结果如下:

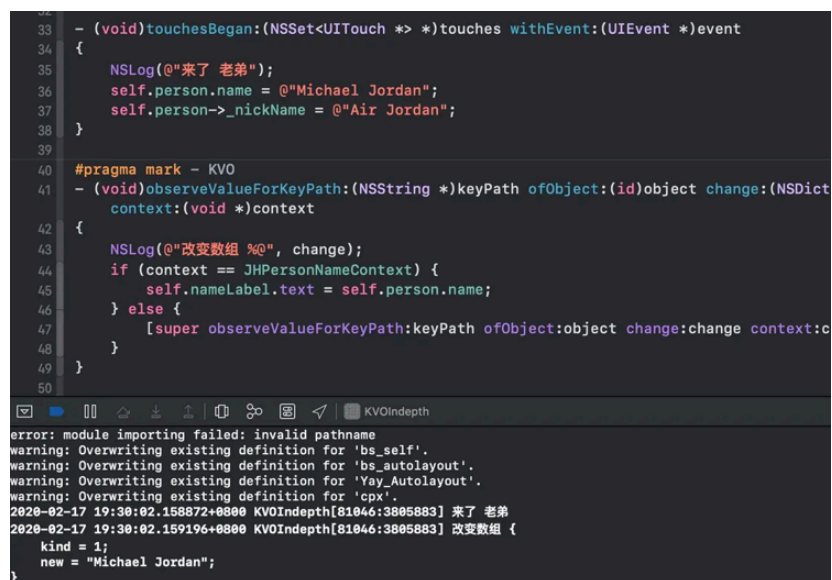
```
classes = (
    JHPerson
)
classes = (
    JHPerson,
    "NSKeyValueObserving_JHPerson"
)
```

结果很清晰, 中间类 `NSKeyValueObserving_JHPerson` 是作为原始真正的类 `JHPerson` 的子类的角色。

2.2 KVO 观察的是什么?

KVO 所关注的是属性值的变化，而属性值本质上是成员变量+getter+setter，getter 是用来获取值的，而显然只有 setter 和成员变量赋值两种方式可以改变属性值。我们测试一下这两种方式：

```
// JHPerson.h
@interface JHPerson : NSObject {
    @public
    NSString *_nickName;
}
@property (nonatomic, copy) NSString *name
@end
```



如上图所示，setter 方法对属性 name 做了修改被 KVO 监听到了，而成员变量 _nickName 的修改并没有被监听到，说明 KVO 底层其实观察的是 setter 方法。

2.3 中间类重写了哪些方法？

我们可以通过打印原始类和中间类的方法列表来验证：

```

30 [self printClassAllMethod:[JHPerson class]];
31 [self.person addObserver:self forKeyPath:@"name" options:NSKeyValueObservingOptionNew context:JHPersonNameContext];
32 NSLog(@"NSKVO notifying JHPerson 中间类中的方法:");
33 [self printClassAllMethod:NSClassFromString(@"NSKVO notifying_JHPerson")];
34 }

```

error: module importing failed: invalid path name
warning: Overwriting existing definition for 'bs_self'.
warning: Overwriting existing definition for 'bs_autolayout'.
warning: Overwriting existing definition for 'JHPerson'.
warning: Overwriting existing definition for 'JHPerson'.
020-02-17 19:41:49.975655+0800 KVOInDepth[81182:3817375] JHPerson 类中的方法:
020-02-17 19:41:49.975782+0800 KVOInDepth[81182:3817375] =====
020-02-17 19:41:49.975891+0800 KVOInDepth[81182:3817375] -[JHPerson dealloc] 0x1029fd9a0
020-02-17 19:41:49.975982+0800 KVOInDepth[81182:3817375] name-0x1029fd978
020-02-17 19:41:49.976098+0800 KVOInDepth[81182:3817375] setName-0x1029fd9f8
020-02-17 19:41:49.976179+0800 KVOInDepth[81182:3817375] willChangeValueForKey-0x1029fd958
020-02-17 19:41:49.976284+0800 KVOInDepth[81182:3817375] didChangeValueForKey-0x1029fd968
020-02-17 19:41:49.976555+0800 KVOInDepth[81182:3817375] NSKVO notifying_JHPerson 中间类中的方法:
020-02-17 19:41:49.976551+0800 KVOInDepth[81182:3817375] =====
020-02-17 19:41:49.976767+0800 KVOInDepth[81182:3817375] setName-0x7fff25721c78
020-02-17 19:41:49.976864+0800 KVOInDepth[81182:3817375] class-0x7fff2572873d
020-02-17 19:41:49.976957+0800 KVOInDepth[81182:3817375] dealloc-0x7fff25728a2
020-02-17 19:41:49.977056+0800 KVOInDepth[81182:3817375] _isKVOA-0x7fff2572849a

`printClassAllMethod` 方法实现如下:

```

- (void)printClassAllMethod:(Class)cls{
    NSLog(@"*****");
    unsigned int count = 0;
    Method *methodList = class_copyMethodList(cls, &count);
    for (int i = 0; i < count; i++) {
        Method method = methodList[i];
        SEL sel = method_getName(method);
        IMP imp = class_getMethodImplement(cls, sel);
        NSLog(@"%@-%p", NSStringFromSelector(sel), imp);
    }
    free(methodList);
}

```

可以看到如上图所示, 原始类和中间类都有 `setter` 方法。根据我们前面所探索的消息发送以及转发流程, 这里的中间类应该是重写了 `setName:`、`class`、`dealloc` 和 `_isKVOA` 方法。

由我们上一小节的测试结果可知, 中间类重写的 `class` 方法结果仍然是返回的是原始类, 显然系统这样做的目的就是隐藏中间类的存在, 让调用者调用 `class` 方法结果前后一致。

2.4 KVO 中间类何时指回去?

我们推断 KVO 注册观察者到移除观察者这一个流程里面, 被观察对象的 `isa` 指针才会指向中间类, 我们用代码测试一下:


```
36 - (void)dealloc
37 {
38     [self.person removeObserver:self forKeyPath:@"name" context:JHPersonNameContext];
39 }
40

(lldb) p _person.isa
(Class) $0 = NSKVONotifying_JHPerson
Fix-it applied, fixed expression was:
_person->isa
(lldb)
```

```
36 - (void)dealloc
37 {
38     [self.person removeObserver:self forKeyPath:@"name" context:JHPersonNameContext];
39 }
40

(lldb) p _person.isa
(Class) $0 = NSKVONotifying_JHPerson
Fix-it applied, fixed expression was:
_person->isa
(lldb) p _person.isa
(Class) $1 = JHPerson
Fix-it applied, fixed expression was:
_person->isa
(lldb)
```

由上图可知，观察者的 `dealloc` 方法中的移除观察者之后，对象的 `isa` 指针已经指回了原始的类。那么是不是此时中间类就被销毁了呢，我们不妨打印一下此时原始类的所有子类信息：

```
36 - (void)dealloc
37 {
38     [self.person removeObserver:self forKeyPath:@"name" context:JHPersonNameContext];
39 }
40

(lldb) p _person.isa
(Class) $0 = NSKVONotifying_JHPerson
Fix-it applied, fixed expression was:
_person->isa
(lldb) p _person.isa
(Class) $1 = JHPerson
Fix-it applied, fixed expression was:
_person->isa
(lldb)
```

结果表明中间类仍然存在，也就是说移除观察者并不会导致中间类销毁，显然这样对于多次添加和移除观察者来说性能上更好。

2.5 KVO 调用顺序

而我们前面说了，有一个中间类的存在，既然要生成中间类，肯定是有意义的，我们梳理一下整个 `KVO` 的流程，从注册观察者到观察者的回调通知，既然有回调通知，那么肯定是在某个地方发出回调的，而由于中间类是不能编译的，所以我们对中间类的父类也就是 `JHPerson` 类，我们重写一下相应的 `setter` 方法，我们不妨测试一下：

```
// JHPerson.m
- (void)setName:(NSString *)name
{
    _name = name;
}

- (void)willChangeValueForKey:(NSString *)key
{
    [super willChangeValueForKey:key];
    NSLog(@"willChangeValueForKey");
}

- (void)didChangeValueForKey:(NSString *)key
{
    NSLog(@"didChangeValueForKey - begin")
    [super didChangeValueForKey:key];
    NSLog(@"didChangeValueForKey - end");
}
```

打印结果如下：

```
2020-02-17 19:34:32.340365+0800 KVOInDepth[81092:3810163] willChangeValueForKey
2020-02-17 19:34:32.340400+0800 KVOInDepth[81092:3810163] didChangeValueForKey - begin
2020-02-17 19:34:32.340691+0800 KVOInDepth[81092:3810163] 监听到了<JHPerson: 0x6000024ade80>的name属性发生了变化(
    kind = 1;
    new = "Michael Jordan";
)
2020-02-17 19:34:32.340881+0800 KVOInDepth[81092:3810163] didChangeValueForKey - end
```

也就是说 KVO 的调用顺序是：

- 调用 `willChangeValueForKey:`
- 调用原来的 `setter` 实现
- 调用 `didChangeValueForKey:`

也就是说 `didChangeValueForKey:` 内部必然是调用了 `observer` 的 `observeValueForKeyPath:ofObject:change:context:` 方法。

三、自定义 KVO 如何实现

我们已经初步了解了 KVO 底层原理，接下来我们尝试自己简单实现一下 KVO。

我们直接跳转到

`addObserver:forKeyPath:options:context:` 方法的声明处：

```
@interface NSObject(NSKeyValueObserverRegistration)

/* Register or deregister as an observer of the value at a key path relative to the receiver. The options determine what is
   included in observer notifications and when they're sent, as described above, and the context is passed in observer
   notifications as described above. You should use -removeObserverForKeyPath:context: instead of -removeObserverForKeyPath:
   whenever possible because it allows you to more precisely specify your intent. When the same observer is registered for the
   same key path multiple times, but with different context pointers each time, -removeObserverForKeyPath: has to guess at
   the context pointer when deciding what exactly to remove, and it can guess wrong.
*/
- (void)addObserver:(NSObject *)observer forKeyPath:(NSString *)keyPath options:(NSKeyValueObservingOptions)options
    context:(nullable void *)context;
- (void)removeObserver:(NSObject *)observer forKeyPath:(NSString *)keyPath context:(nullable void *)context
    API_AVAILABLE(macos(10.7), ios(5.0), watchos(2.0), tvos(9.0));
- (void)removeObserver:(NSObject *)observer forKeyPath:(NSString *)keyPath;

@end
```

可以看到，跟 `KVC` 一样，`KVO` 在底层也是以分类的形式加载的，这个分类叫做

`NSKeyValueObserverRegistration`。我们不妨也以这种方式来自定义实现一下 `KVO`。

```
// NSObject+JHKVO.h
@interface NSObject (JHKVO)
// 观察者注册
- (void)jh_addObserver:(NSObject *)observer
// 回调通知观察者
- (void)jh_observeValueForKeyPath:(nullable void *)context
// 移除观察者
- (void)jh_removeObserver:(NSObject *)observer
@end
```

这里为了避免与系统的方法冲突，所以添加了一个方法前缀。同时对于观察策略，为了简化实现，这里只声明了新值和旧值两种策略。

3.1 自定义观察者注册

在开始之前，我们回忆下自定义 `KVC` 的时候的第一个步骤就是判断 `key` 或者 `keyPath`，那么 `KVO` 是否也需要进行这样的判断呢？经过笔者实际测试，如果观察对象的一个不存在的属性的话，并不会报错，也不会来到 `KVO` 回调方法，由此可见，判断 `keyPath` 是否存在并没有必要。但是，我们回想一下上一节 `KVO` 底层

原理，KVO 关注的是属性的 setter 方法，那其实判断对象所属的类是否有这样的 setter 就相当于同时判断了 keyPath 是否存在。接着我们就需要去动态的创建子类，创建子类的过程中包括了重写 setter 等一系列方法。然后就需要保存观察者和 keyPath 等信息，这里我们借助关联对象来实现，我们把传入的观察者对象、keyPath 和观察策略封装成一个新的对象存储在关联对象中。因为同一个对象的属性可以被不同的观察者所观察，所以这里实质上是以对象数组的方式存储在关联对象里面。

话不多说，直接上代码：

```
// JHKVOInfo.h
typedef NS_OPTIONS(NSUInteger, JHKeyValueC
    JHKeyValueObservingOptionNew = 0x01,
    JHKeyValueObservingOptionOld = 0x02,
};
@interface JHKVOInfo : NSObject
@property (nonatomic, weak) NSObject *obs
@property (nonatomic, copy) NSString *key
@property (nonatomic, assign) JHKeyValueOb
- (instancetype)initWithObserver:(NSObject
@end

// JHKVOInfo.m
@implementation JHKVOInfo
- (instancetype)initWithObserver:(NSObject
{
    if (self = [super init]) {
        _observer = observer;
        _keyPath = keyPath;
        _options = options;
    }
    return self;
}
@end
```

上面的代码是自定义的 JHKVOInfo 对象。

```

Method setterMethod = class_getInsta
if (!setterMethod) {
    NSString *reason = [NSString str
    @throw [NSException exceptionWit
        r
        use

    return;
}

// 2.动态创建中间子类
Class newClass = [self createChildCl

// 3.将对象的isa指向为新的中间子类
object_setClass(self, newClass);

// 4.保存观察者
JHKVOInfo *info = [[JHKVOInfo alloc]
NSMutableDictionary *observerArr = objc_g

if (!observerArr) {
    observerArr = [NSMutableDictionary ar
    [observerArr addObject:info];
    objc_setAssociatedObject(self, (
}
}

```

上面的代码是完整的添加观察者的流程：

- 判断对象所属的类上是否有要观察的 `keyPath` 对应的 `setter` 方法

这里的 `setterForGetter` 实现如下：

```

static NSString * setterForGetter(NSString:
{
    // 判断 getter 是否为空字符串
    if (getter.length <= 0) {
        return nil;
    }
    // 取出 getter 字符串的第一个字母并转大写

```

```
NSString *firstLetter = [[getter sub:  
// 取出剩下的字符串内容  
NSString *remainingLetters = [getter  
// 将首字母大写的字母与剩下的字母拼接起来得  
NSString *setter = [NSString stringWith:  
return setter;  
}
```

- 如果存在相应的 `setter` 方法，那么就创建有对应前缀的中间子类

这里的 `createChildClassWithKeyPath` 实现如下：

```
- (Class)createChildClassWithKeyPath:(  
// 获得原始类的类名  
NSString *oldClassName = NSStringF  
// 在原始类名前添加中间子类的前缀来获得  
NSString *newClassName = [NSString  
// 通过中间子类名来判断是否创建过  
Class newClass = NSClassFromString  
// 如果创建过中间子类，直接返回  
if (newClass) return newClass;  
// 如果没有创建过，则需要创建一下，objc  
newClass = objc_allocateClassPair(  
// 注册中间子类  
objc_registerClassPair(newClass);  
// 从父类上拿到 `class` 方法的 `SEL`  
SEL classSEL = NSSelectorFromStrin  
Method classMethod = class_getInst  
const char *classTypes = method_ge  
class_addMethod(newClass, classSEL  
// 从父类上拿到 `getter` 方法的 `SEL`  
SEL setterSEL = NSSelectorFromStri  
Method setterMethod = class_getIns  
const char *setterTypes = method_g  
class_addMethod(newClass, setterSE  
return newClass;  
}
```

`jh_class` 的实现如下:

```
Class jh_class(id self,SEL _cmd) {
    // 通过 class_getSuperclass 来返回父类的
    return class_getSuperclass(object_getClass(self));
}
```

`jh_setter` 的实现如下:

```
static void jh_setter(id self,SEL _cmd, id value) {
    // 因为 `_cmd` 作为方法的第二个参数其实是指针
    NSString *keyPath = getterForSetter(self, _cmd);
    id oldValue = [self valueForKeyPath:keyPath];

    // 因为是重写父类的 `setter`, 所以还需调用父类的 `setter`
    // 通过强转的方式将 `objc_msgSendSuper` 转成 `void (*)(void *, SEL, void *)`
    void (*jh_msgSendSuper)(void *, SEL, void *) = (void (*)(void *, SEL, void *))objc_msgSendSuper;
    struct objc_super superStruct = {
        .receiver = self,
        .super_class = class_getSuperclass([self class]);
    };
    // 准备工作完成后手动调用 `jh_msgSendSuper`
    jh_msgSendSuper(&superStruct, _cmd, value);
    // 调用完父类的 `setter` 之后, 从关联对象中移除观察者
    NSMutableArray *observerArr = objc_getAssociatedObject(self, &objc_AssociatedKey_JHKeyValueObserving);
    // 循环遍历自定义的对象
    for (JHKVOInfo *info in observerArr) {
        // 如果 `keyPath` 匹配则进入下一步
        if ([info.keyPath isEqualToString:keyPath]) {
            // 基于线程安全的考虑, 使用 `GCD`
            dispatch_async(dispatch_get_main_queue(), ^{
                // 初始化一个通知字典
                NSMutableDictionary<NSString, id> *dict = [NSMutableDictionary dictionary];
                // 判断存储的观察策略, 如: 立即通知
```

`getterForSetter` 实现如下:

```
static NSString *getterForSetter(NSString *keyPath, SEL _cmd) {
```

```

// 判断传入的 `setter` 字符串长度是否大于 3
if (setter.length <= 0 || ![setter l
// 排除掉 `setter` 字符串中的 `set:` 部
NSRange range = NSMakeRange(3, setter
NSString *getter = [setter substring
// 对 getter 字符串首字母小写处理
NSString *firstString = [[getter sul
return [getter stringByReplacingCha
}

```

3.2 自定义移除观察者

我们接着开始自定义移除观察者，首先，我们需要把 `isa` 指回原来的类，然后需要对关联对象中存储的自定义对象数组对应的观察者移除掉。

```

- (void)jh_removeObserver:(NSObject *)obj
{
    // 从关联对象中取出数组
    NSMutableArray *observerArr = objc_g
    // 如果数组中没有内容，说明没有添加过观察者
    if (observerArr.count<=0) {
        return;
    }

    // 遍历取出的所有自定义对象
    for (JHKVOInfo *info in observerArr)
        // 如果 `keyPath` 匹配上了 则从数组中
        if ([info.keyPath isEqualToString
            [observerArr removeObject:info
            objc_setAssociatedObject(self,
            break;
        }
    }

    // 要将 `isa` 指回原来的类的前提条件是，被
    if (observerArr.count<=0) {
        Class superClass = [self class];
        object_setClass(self, superClass
    }
}

```



```
}
```

3.3 实现自动移除观察者

现在我们自定义的 **KVO** 已经可以实现简单的通知观察者新值和旧值的变化了，但其实对于 **api** 的使用者来说，还是要严格的执行 **addObserver** 和 **removeObserver** 的配套操作，难免有些繁琐。虽然一般来说为了方便起见，都是在观察者的 **dealloc** 方法中去手动调用 **removeObserver** 方法，但还是太麻烦了。因此，我们可以借助 **methodSwizzling** 的技术来替换默认 **dealloc** 方法的实现，直接上代码：

```
+ (BOOL)jh_hookOrigInstanceMenthod:(SEL)
// 获取 Class 对象
Class cls = self;
// 通过 `SEL` 获取原始方法
Method oriMethod = class_getInstance
// 通过 `SEL` 获取要替换的方法
Method swiMethod = class_getInstance
// 如果要替换的方法不存在，返回 NO
if (!swiMethod) {
    return NO;
}
// 如果原始方法不存在，那么就直接在 Class
if (!oriMethod) {
    class_addMethod(cls, oriSEL, met
    method_setImplementation(swiMeth
}

// 判断是否添加成功
BOOL didAddMethod = class_addMethod(
if (didAddMethod) {
// 如果成功，说明 Class 上已经存在了要替换
    class_replaceMethod(cls, swizzle
}else{
// 如果不成功，说明原始方法已经存在，则直接
    method_exchangeImplementations(o
}
```

通过实现自动移除观察者，`api` 的使用者可以完全放心的只使用 `addObserver` 来添加观察者以及 `observeValueForKeyPath` 来接收回调。

3.4 函数式编程思想重构

我们虽然已经实现了自动的移除观察者，但是从函数式编程思想来看，现在的设计还不是很完美，对同一个属性的观察的代码散落在不同的地方，如果业务一旦增多，对于可读性和可维护性都有很大的影响。所以，我们可以把现在这种回调的形式重构为 `Block` 的方式。

```
// NSObject+JHBlockKVO.h
typedef void(^JHKVBlock)(id observer, NS
@interface NSObject (JHBlockKVO)
- (void)jh_addObserver:(NSObject *)obser
- (void)jh_removeObserver:(NSObject *)ob
@end

// NSObject+JHBlockKVO.m
@interface JHBlockKVOInfo : NSObject
@property (nonatomic, weak) NSObject *
@property (nonatomic, copy) NSString *
@property (nonatomic, copy) JHKVBlock
@end

@implementation JHBlockKVOInfo
- (instancetype)initWithObserver:(NSObjec
    if (self=[super init]) {
        _observer = observer;
        _keyPath = keyPath;
        _handleBlock = block;
    }
    return self;
}
@end

@implementation NSObject (JHBlockKVO)
```

这里我们直接通过传入分类一个 `block`，然后存储在对应的自定义观察对象中，然后我们还需要在重写 `setter` 方法中做出修改，原来是直接通过发送消息来实现回调，现在需要改成 `block` 回调

```
static void jh_setter(id self, SEL _cmd, id
NSString *keyPath = getterForSetter(NS
id oldValue = [self valueForKey:keyPat
void (*jh_msgSendSuper)(void *, SEL , i
struct objc_super superStruct = {
    .receiver = self,
    .super_class = class_getSuperclass
};
jh_msgSendSuper(&superStruct, _cmd, newV

NSMutableArray *mArray = objc_getAssoc
for (JHBlockKVOInfo *info in mArray) {
    if ([info.keyPath isEqualToString:
        info.handleBlock(info.observer
    }
}
}
```

四、总结

经过探索 `KVC` 和 `KVO` 的底层，我们可以看到 `KVO` 是建立在 `KVC` 基础之上的。`KVO` 作为观察者设计模式在 `iOS` 中的具体落地，其原理到实现我们都探索完了。其实我们可以看出来在早期设计 `api` 的时候，原生的 `KVO` 其实并不好用，所以诸如 `FaceBook` 的库 `KVOCotroller` 会大受欢迎。当然本文的自定义 `KVO` 实现并不严谨，感兴趣的读者可以查看这两个代码库：

- 根据原生的 `KVC` 和 `KVO` 反汇编而成 `DIS_KVC_KVO`
- 开源的 `GNUStep` 的 `libs-base gnustep/libs-base`

我们的 **iOS** 底层探索系列接下来将会进入多线程篇章，敬请期待~

参考资料

[Key-Value Observing Programming Guide - Apple 官方文档](#)

[nil/Nil/Null/NSNull - NSHipster](#)

[Key-Value Observing - NSHipster](#)

 [ios](#) [objective-c](#) [xcode](#) [swift](#)  [flutter](#)

 赞 1

 收藏 1

 分享

阅读 3k · 发布于 2020-02-24



leejunhui

95 声望 · 14 粉丝 · 

Never quit on the half way of pursuing perfect.

[关注作者](#)

« 上一篇

[iOS 底层探索 - KVC](#)

下一篇 »

[iOS 查漏补缺 - ...](#)

引用和评论

推荐阅读

**iOS 查漏补缺 - LLVM & Clang**

leejunhui · 赞 1 · 阅读 4k

**Flutter 3.29 中有什么新内容**

独立开发者_猫哥 · 阅读 1.4k

**MacOS15+Xcode版本16+对
ReactNative项目进行编译和上传到
APPStore的踩坑记录**

kovli · 阅读 938

**微信分享前端开发全程详解含iOS、安
卓、H5、ReactNative以及微信开放标
签的适配和使用**

kovli · 阅读 931

**用 Cursor AI 写 flutter 直接喂设计图就
行**

独立开发者_猫哥 · 赞 1 · 阅读 915

**flutter3-trip-hotel: 2025实战
Flutter3.27仿携程旅行App酒店预订系
统模板**

xiaoyan2017 · 阅读 893

**通义千问2.5-Max + Roo Code Cline 插
件: 实现 AI Agents 自动编程。**

独立开发者_猫哥 · 阅读 820

0 条评论

得票最新



撰写评论 ...



提交评论

评论支持部分 Markdown 语法: ****粗体****
斜体 [链接](http://example.com) `代码` - 列表 > 引用。你还可以使用 @ 来通知其他用户。

©2025 iOSDaily

除特别声明外，作品采用《署名-非商业性使用-禁止演绎 4.0 国际》进行许可

 使用 SegmentFault 发布

SegmentFault - 凝聚集体智慧，推动技术进步

服务协议 · 隐私政策 · 浙ICP备15005796号-2 · 浙公网安备33010602002000号